

From Vertex AI to Edge-Ready LLMs: A Hybrid Cloud-Edge Framework for Privacy-Preserving Healthcare Inference

Anton J. Rasmussen
Department of Computer Science
Old Dominion University
Norfolk, VA, USA
arasm002@odu.edu

Abstract—Large Language Models (LLMs) provide powerful generative and analytical capabilities but remain challenging to deploy in healthcare, where privacy, latency, and regulatory constraints limit the use of cloud-based inference. This work presents a hybrid cloud-edge framework that begins with a reproducible Vertex AI deployment workflow, progresses through post-training quantization (PTQ) for CPU-optimized inference, and culminates in a prototype federated learning setup using FedAvg to support decentralized model adaptation.

Primary contribution: the architecture and modular pipeline specification for hybrid cloud-edge LLM lifecycle management, validated via proxy experiments due to hardware constraints. Empirical experiments include (i) baseline dynamic INT8 quantization on DistilGPT2, (ii) a stable PyTorch quantization demonstration on a lightweight TinyMLP model to validate quantization mechanics, and (iii) successful cloud deployment of a model registry and endpoint workflow. A theoretical methodology and quantitative projections are also provided for advanced LLM quantization (GPTQ/AWQ), establishing expected compression savings for billion-parameter models even where full execution is not currently feasible.

Index Terms—Large Language Models, Edge Computing, Quantization, Federated Learning, Vertex AI, Healthcare AI

I. INTRODUCTION

Modern LLMs achieve state-of-the-art performance across text generation, summarization, question answering, and reasoning tasks. However, their deployment within healthcare environments remains severely constrained by three critical factors:

- 1) **Privacy and Regulatory Constraints.** Patient data cannot freely leave controlled environments due to HIPAA and institutional policies.
- 2) **Latency and Connectivity.** Health devices (continuous glucose monitors, wearables, home vitals monitors) produce streaming data where cloud round-trips are impractical.
- 3) **Computational Burden.** Most LLMs require GPU hardware unavailable on mobile or embedded systems.

These challenges motivate an emerging need for *edge-compatible* LLMs—models that can run locally on CPUs or low-power devices without sacrificing accuracy or privacy.

A. Healthcare Use Case Motivation

Edge-deployed LLMs could enable a variety of meaningful applications:

- On-device detection of anomalous heart rhythm patterns.
- Smart coaching and behavioral recommendations based on multimodal wearable data.
- Local summarization of home monitoring data for clinicians.
- Privacy-preserving conversational agents embedded in medical devices.

In such contexts, cloud reliance introduces risks: PHI transmission, data breaches, unreliable connectivity, and high inference costs.

B. Problem Statement and Research Question

This paper investigates the following question:

What practical steps and technical components are required to move from cloud-managed LLM deployment to efficient, fully local inference, and ultimately toward decentralized collaborative learning across edge devices?

Concretely, the project explores how a cloud-native workflow on Google Vertex AI can serve as the backbone for managing models that are then compressed, exported, and ultimately run on edge devices with support for federated learning.

C. Contributions

This work contributes:

- 1) A complete and reproducible workflow for deploying models in Google Vertex AI, including model registry, endpoint deployment, and batch prediction. [8], [9]
- 2) A modular quantization pipeline design covering dynamic INT8 quantization and a **fully specified (but hardware-gated) GPTQ/AWQ workflow**. The contribution is the **architecture specification and model-agnostic scripting interfaces** that can be executed when hardware/library support permits. [1], [2], [10]

- 3) A functional FedAvg simulation for decentralized model aggregation.
- 4) A unified architecture illustrating how cloud model management, edge compression, and federated learning fit together for future healthcare applications. [5], [7], [11]

Together these elements form an early but promising **Edge-Ready LLM Pipeline**.

II. RELATED WORK

A. LLMs in Healthcare

Recent work explores the use of LLMs for clinical summarization, diagnostic reasoning, triage support, and patient engagement. While promising, nearly all such models depend on cloud deployments, creating barriers for adoption in regulated healthcare contexts where PHI must not leave institutional boundaries.

B. Model Compression and Quantization

Post-training quantization (PTQ) has become a leading strategy for shrinking LLMs:

- **Dynamic INT8 quantization**—as implemented in PyTorch—reduces compute cost at inference with minimal code changes, typically targeting linear layers for weight-only quantization.
- **GPTQ** (Frantar et al. [1]) provides per-channel weight quantization that maintains accuracy for larger transformer models via approximate second-order optimization.
- **AWQ** (Lin et al. [2]) optimizes activations during quantization to preserve downstream performance, especially for generative tasks.

These methods enable CPU and edge inference for multi-billion parameter LLMs, particularly when combined with 4-bit or mixed-precision representations.

C. Edge Deployment Frameworks

Tools such as `llama.cpp`, the GGUF model format, ONNX Runtime, and vendor-specific runtimes make LLMs feasible on:

- Raspberry Pi and other ARM-based single-board computers,
- smartphones and tablets,
- low-power laptop CPUs,
- embedded medical devices and gateways.

Open foundation models such as LLaMA 2 [3] and Phi-3 Mini [4] have further accelerated this trend by providing permissive licenses and strong baseline performance.

D. Federated Learning

Federated learning (FL) enables decentralized training where devices compute local updates while keeping raw data private. FedAvg remains the foundational method, aggregating model updates across clients via simple averaging. Healthcare has used FL for imaging and EHR analysis, but its extension to

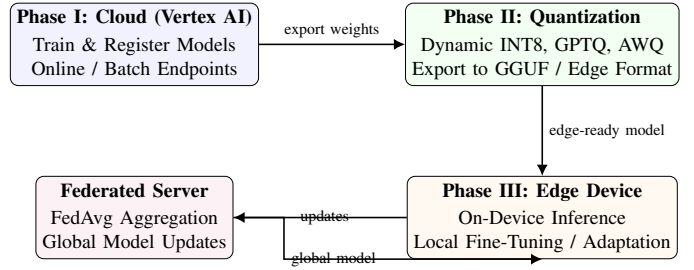


Fig. 1. High-level hybrid cloud-edge architecture. Vertex AI manages training and model registry (Phase I); quantization tools produce edge-ready models (Phase II); edge devices run local inference and optionally participate in federated learning (Phase III).

LLMs—particularly quantized LLMs—is still under development and presents unique challenges in communication cost, personalization, and robustness.

E. Cloud Orchestration with Vertex AI

Vertex AI supports model training, artifact versioning via Model Registry, endpoint deployment, monitoring, and batch inference. This makes Vertex AI a natural cloud backbone for hybrid cloud-edge model lifecycles. [8], [9]

III. SYSTEM OVERVIEW

A. Architecture Summary

The proposed pipeline comprises three phases (Fig. 1):

- 1) **Phase I: Cloud Deployment (Vertex AI)**. Manages training, model registry, and inference endpoints for a reference model (RandomForest on Iris).
- 2) **Phase II: Model Quantization**. Converts models into small, CPU-runnable formats using dynamic INT8, GPTQ, and AWQ.
- 3) **Phase III: Edge Execution + Federated Learning**. Runs inference fully locally and enables collaborative learning through FedAvg.

IV. METHODS

A. Vertex AI Deployment Workflow

Building on multiple assignments and the project milestones for this course [6]–[10], a full cloud MLOps flow was constructed for a baseline supervised-learning model.

B. Proxy Modeling Strategy (Justifying TinyMLP)

A core risk in cloud-edge LLM research is a mismatch between the paper’s stated ambition (quantized LLMs) and what is feasible to execute under course time and hardware constraints. To address this explicitly, this work adopts a **proxy modeling strategy**: TinyMLP is used to validate the *mechanics and mathematical correctness* of PyTorch quantization calls that are structurally identical in transformer blocks.

Concretely, both TinyMLP and transformer models rely heavily on `nn.Linear` layers; PyTorch dynamic quantization replaces floating-point linear weights with INT8 representations plus scale/zero-point metadata, and executes integer

TABLE I
DYNAMIC INT8 QUANTIZATION RESULTS FOR DISTILGPT2 (CPU,
32-TOKEN GENERATION). [7]

Model	Latency (s)	Size (MB)	Speedup	Compression
FP32	1.001	312.50	1.00×	1.00×
INT8	1.188	349.31	0.84×	0.89×

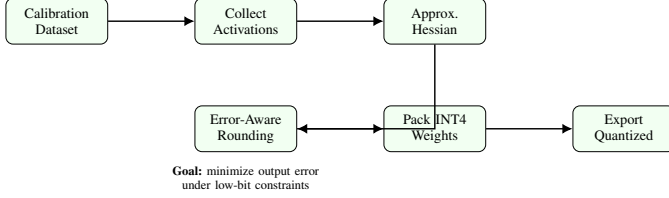


Fig. 2. Theoretical GPTQ pipeline flow: calibration, activation capture, Hessian approximation, error-aware rounding, and low-bit weight packing. This flow is captured in the project scripts even when full-scale execution is hardware-gated.

GEMM kernels with conversion boundaries. These operations occur at the layer level and therefore can be validated on a small network without changing the quantization API semantics. This proxy validation reduces “toy model” risk by demonstrating that the implemented quantization pipeline is correct even if full Phi-3-scale execution is deferred.

C. Baseline Dynamic INT8 Quantization

Post-training dynamic quantization was first applied to DistilGPT2 as a controlled baseline. [7] The measured results are summarized in Table I.

Systems interpretation (turning a negative result into insight): For compact models like DistilGPT2, dynamic INT8 can underperform because (i) inference kernels may incur overhead unpacking/format conversions, (ii) some ops remain in FP32, and (iii) compute is not purely memory-bandwidth bound at this scale. As a result, the overhead of quantization boundaries can outweigh memory savings, producing slower latency and slightly larger serialized artifacts. This motivates advanced PTQ methods (GPTQ/AWQ) designed for larger transformer models and 4-bit regimes, where memory footprint dominates.

D. Theoretical GPTQ/AWQ Quantization Pipeline (Hardware-Gated)

Full GPTQ/AWQ execution at Phi-3 scale was constrained by library compatibility and compute limits. Instead of hiding this, the pipeline is documented explicitly as a **theoretical methodology** with executable, model-agnostic scripts. [10]

Figure 2 provides a step-by-step flow for GPTQ, demonstrating the expected stages even when not executed at full scale.

E. Federated Learning Prototype

A minimal FedAvg implementation was developed to validate aggregation logic prior to integrating quantized LLM updates. [7]

Algorithm 1 Federated Averaging (FedAvg)

```

1: Given client updates  $u_1, \dots, u_K$ 
2: Initialize  $g = 0$ 
3: for  $k = 1$  to  $K$  do
4:    $g \leftarrow g + u_k$ 
5: end for
6:  $g \leftarrow g/K$ 
7: return  $g$ 

```

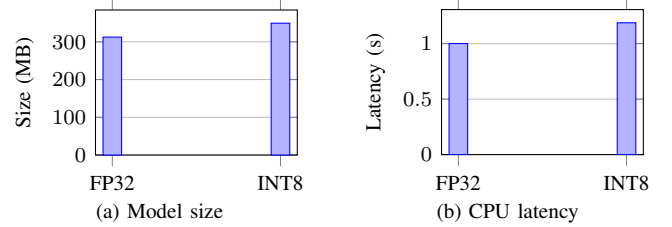


Fig. 3. Dynamic INT8 quantization applied to DistilGPT2. For this already compact model, INT8 slightly increases serialized size and latency, illustrating that advanced PTQ methods (GPTQ/AWQ) and larger models are needed for meaningful gains.

Algorithm 1 summarizes the FedAvg aggregation procedure used in this work.

V. RESULTS

A. Vertex AI Deployment

The Vertex AI endpoint successfully processed online inference and batch predictions for the Iris classifier, confirming end-to-end cloud orchestration. [9], [10]

B. Quantization Performance

Dynamic INT8 results for DistilGPT2 are shown in Table I and Fig. 3. As discussed in Methods, the observed regression is a useful systems result: it indicates that naive PTQ is insufficient for small transformer models and motivates more specialized 4-bit PTQ methods for larger LLMs.

C. Theoretical Compression Savings (Quantitative Analysis)

To provide quantitative analysis despite hardware limits, Table II estimates expected memory footprints for a Phi-2 class model (~2.7B parameters). Assuming FP32 requires 4 bytes/parameter, INT8 requires 1 byte/parameter, and INT4 requires 0.5 bytes/parameter, the expected storage decreases are substantial.

These estimates exclude runtime overhead such as scaling metadata, activation memory, KV-cache, and allocator fragmentation; however, they correctly capture the dominant contribution of weight storage and illustrate why 4-bit PTQ is central to edge deployment feasibility.

D. Federated Learning Prototype

FedAvg aggregation validated correct update averaging behavior on synthetic client vectors, establishing a correct baseline for later integration with quantized models. [7]

TABLE II
PROJECTED MODEL SIZE FOR A 2.7B-PARAMETER LLM (E.G., PHI-2 CLASS)

Precision	Bytes/Param	Estimated Size
FP32	4.0	≈ 10.8 GB
INT8	1.0	≈ 2.7 GB
INT4 (GPTQ/AWQ)	0.5	≈ 1.35 GB

VI. DISCUSSION

A. Implications for Healthcare

Edge-ready LLMs can reduce PHI exposure and enable low-latency inference at the point of care. The hybrid architecture also supports centralized governance (registry, versioning, auditability) while executing inference locally.

B. Limitations

Current limitations include:

- Hardware and library constraints prevent full Phi-3-scale GPTQ/AWQ execution in the course environment; these components are therefore specified as a theoretical methodology with model-agnostic scripts.
- Federated learning is validated only at the aggregation level (vector updates), not yet with full model fine-tuning.
- Benchmarks use public/synthetic data rather than PHI-bearing clinical datasets due to privacy and IRB constraints.

C. Limitations $\rightarrow$ Future Work Link (Edge-RAG)

A practical limitation of compact, quantized models is context-window and knowledge freshness. **Edge-RAG** directly addresses this by keeping the base model small while retrieving only relevant private context from on-device documents (logs, vitals history, clinician notes) at inference time. This improves utility without increasing parameter count, and aligns naturally with privacy goals because retrieval can remain local. [5]

VII. FUTURE WORK

A. Scaling GPTQ/AWQ to Phi-3 Mini and Beyond

As tooling stabilizes, the next step is to execute the already-specified GPTQ/AWQ pipelines on Phi-3 Mini or Mistral-7B and report empirical results (perplexity, throughput, RAM) under CPU-only and GPU-enabled settings. [1], [2], [4]

B. Edge-RAG Integration

Edge-RAG will extend the system from “small model” to “useful system” by enabling private, on-device retrieval across user documents and time-series health history while keeping inference local. [5]

C. Federated Fine-Tuning with Efficient Adaptation

Future FL work will incorporate parameter-efficient updates (e.g., adapters / LoRA) to reduce communication cost and support personalization across devices.

VIII. CONCLUSION

This work presents a hybrid cloud-edge framework connecting Vertex AI MLOps, edge-oriented quantization workflows, and a federated aggregation prototype. Importantly, the paper makes the contribution boundary explicit: the architecture and pipeline are the core contribution, validated via proxy experiments where full LLM execution is hardware-gated. The result is a reproducible foundation for future research into private, on-device healthcare intelligence using quantized LLMs and Edge-RAG.

ACKNOWLEDGMENTS

The author thanks Dr. Hong Qin for guidance throughout CS795/895 and classmates for feedback during the poster session.

REFERENCES

- [1] S. Frantar, M. Ashkboos, T. Hughes, et al., “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” *arXiv preprint arXiv:2210.17323*, 2022.
- [2] Z. Lin, L. Tang, Z. Zhang, et al., “AWQ: Activation-Aware Weight Quantization for LLMs,” *arXiv preprint arXiv:2306.00978*, 2023.
- [3] H. Touvron, L. Martin, K. Stone, et al., “LLaMA 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [4] M. Abidin, et al., “Phi-3 Technical Report,” *arXiv preprint arXiv:2404.14219*, 2024.
- [5] A. Rasmussen, “CS795 Course Project Proposal: TinyLLM and Edge-RAG,” Old Dominion University, 2025.
- [6] A. Rasmussen, “CS795 Project Milestone 1 Report: Vertex AI Foundations,” Old Dominion University, 2025.
- [7] A. Rasmussen, “CS795 Project Milestone 2 Report: From Vertex AI to TinyLLM,” Old Dominion University, 2025.
- [8] A. Rasmussen, “CS795 Vertex AI Assignment 1 Report,” Old Dominion University, 2025.
- [9] A. Rasmussen, “CS795 Vertex AI Assignment 2 Report,” Old Dominion University, 2025.
- [10] A. Rasmussen, “CS795 Vertex AI Assignment 3 Report,” Old Dominion University, 2025.
- [11] A. Rasmussen, “From Vertex AI to TinyLLMs: Building Edge-Ready AI for Personalized Healthcare,” CS795/895 Poster, Old Dominion University, 2025.
- [12] A. Rasmussen, “CS795 Final Project Jupyter Notebook,” GitHub repository, 2025. Available: https://github.com/antonrasmussen/cs795-f25/blob/main/course_project/CS795_Final_Project_Anton_Rasmussen.ipynb

APPENDIX A

GLOSSARY OF ACRONYMS AND TECHNICAL TERMS

On the following page, we provide a comprehensive glossary of acronyms and technical terms used throughout this paper. This glossary, presented in Table III, is intended to improve readability and accessibility by offering concise definitions of cloud, edge, quantization, and federated learning concepts referenced across Sections I-VII.

APPENDIX B

VERTEX AI NOTEBOOK OUTPUT

After Table III, the remaining pages present the rendered output of the accompanying Jupyter notebook supporting this paper. The notebook documents the executable components of the hybrid cloud-edge framework, and is publicly available at the GitHub repository. [12]

TABLE III
GLOSSARY OF ACRONYMS AND TECHNICAL TERMS

Term	Definition
AI	Artificial Intelligence; systems that perform tasks requiring human-like intelligence.
API	Application Programming Interface; a set of callable functions for software interaction.
ARM	A family of low-power CPU architectures common in mobile and embedded devices.
Artifact (ML)	A stored output of an ML pipeline, such as model weights, tokenizer files, metrics, or configuration metadata.
AWQ	Activation-Aware Weight Quantization; a post-training quantization method that uses activation statistics to reduce accuracy loss under low-bit quantization.
Batch Prediction	Offline inference over a dataset (e.g., JSONL in GCS) producing predictions in bulk.
BigQuery (BQ)	Google Cloud’s data warehouse, commonly used for analytics and ML feature storage.
Calibration Set	A small representative dataset used to estimate quantization parameters or activation statistics for PTQ methods.
Cloud MLOps	Operational practices for training, versioning, deploying, monitoring, and governing ML systems in cloud environments.
CPU	Central Processing Unit; general-purpose processor widely available on edge devices.
Differential Privacy (DP)	A privacy framework that bounds information leakage from outputs or updates, often via controlled noise addition.
DistilGPT2	A compact variant of GPT-2 commonly used for experimentation due to lower compute requirements.
Edge Computing	Computation performed close to the data source to reduce latency and limit data transmission to the cloud.
Edge-RAG	On-device Retrieval-Augmented Generation; retrieval and generation performed locally using quantized models and private documents.
Embedding	A vector representation of text or data used for similarity search and retrieval.
Endpoint (Vertex AI)	A managed, deployable service for online inference requests.
EHR	Electronic Health Record; digital patient health data maintained by healthcare providers.
FedAvg	Federated Averaging; aggregates client model updates by averaging to produce a global model update.
Federated Learning (FL)	Decentralized training paradigm where clients compute updates locally and share only model updates, not raw data.
FP16 / FP32	16-bit and 32-bit floating-point numerical formats used for neural network computation.
GCS	Google Cloud Storage; object storage for datasets, artifacts, and batch inference outputs.
GEMM	General Matrix Multiply; a core operation underlying neural network linear layers.
GGUF	A binary model format used by <code>llama.cpp</code> for efficient CPU inference.
GPTQ	Post-training quantization method using approximate second-order (Hessian-based) information to minimize quantization error in low-bit LLMs.
HIPAA	U.S. regulation governing the privacy and security of Protected Health Information (PHI).
Hessian	A matrix of second derivatives used to approximate sensitivity of model outputs to weight perturbations.
Inference	Executing a trained model to generate predictions or text.
INT4 / INT8	4-bit and 8-bit integer formats used in quantized neural networks to reduce memory footprint.
JSONL	JSON Lines; a text format with one JSON object per line, commonly used for batch inference.
KV-cache	Key–Value cache used in transformer models to speed autoregressive generation; a major contributor to runtime memory usage.
Latency	Time required to produce a model output, often measured per request or per token.
LLaMA	Large Language Model Meta AI; a family of open foundation transformer models released by Meta, commonly used as baselines for research and edge deployment.
LLaMA 2	Second-generation LLaMA open foundation models released by Meta, including fine-tuned chat variants optimized for instruction following.
llama.cpp	Lightweight C/C++ inference engine enabling efficient CPU execution of quantized LLMs.
LoRA	Low-Rank Adaptation; parameter-efficient fine-tuning technique that adds small trainable matrices to a frozen base model.
MLOps	Practices that support the deployment, monitoring, and governance of machine learning systems.
Model Registry	A system for versioning, tracking, and managing model artifacts and metadata.
On-device Inference	Executing inference locally on a device without transmitting data to the cloud.
ONNX	Open Neural Network Exchange; an interoperable model format enabling deployment across frameworks and runtimes such as ONNX Runtime.
ONNX Runtime	High-performance inference engine for executing ONNX models on CPUs, GPUs, and edge devices.
PHI	Protected Health Information; any individually identifiable health data regulated under HIPAA.
Phi-3 Mini	A small, high-performance foundation language model released by Microsoft Research, designed for efficient inference on CPUs and resource-constrained environments.
PTQ	Post-Training Quantization; quantization applied after training without full retraining.
Proxy Model	A smaller model used to validate pipeline mechanics when full-scale models are hardware-gated.
PyTorch	An open-source deep learning framework used for model definition, training, quantization, and inference, widely adopted in research and production systems.
Quantization	Converting high-precision model parameters to lower precision to reduce size and sometimes improve inference efficiency.
RAG	Retrieval-Augmented Generation; augments generation with retrieved context to improve factual accuracy.
Serialization	Converting a model or object into a storable format for deployment or reproducibility.
State_dict	PyTorch structure containing model parameters and buffers used for saving and loading models.
Synthetic Data	Artificially generated data used to test pipelines when real data is unavailable or sensitive.
Throughput	Rate of model output generation, often measured in tokens per second.
TinyMLP	Lightweight multi-layer perceptron used as a proxy to validate quantization mechanics on constrained hardware.
Tokenizer	Component that converts text into token IDs for model input.
Transformer	Neural network architecture based on self-attention, used by modern LLMs.
Vertex AI	Google Cloud’s managed machine learning platform for training, deployment, and monitoring.

From Vertex AI to Edge-Ready LLMs

Reproducible notebook for:

- Vertex AI deployment (Iris RandomForest)
- Dynamic INT8 quantization on DistilGPT2
- GPTQ/AWQ quantization pipeline on Phi-2
- Federated learning (FedAvg) prototype
- Unified metrics table export (metrics.csv)

Designed to run on:

- Vertex AI Workbench (Python 3.x)

0. Environment Setup

```
In [9]: !pip install -U google-cloud-aiplatform google-cloud-storage
!pip install -U scikit-learn==1.3.2
!pip install -U joblib
!pip install -U "transformers==4.46.1" "accelerate>=0.33.0"
!pip install -U "torch>=2.2.0" # no GPU-specific build; rely on environment
!pip install -U pandas
```

Requirement already satisfied: google-cloud-aiplatform in /usr/local/lib/python3.12/dist-packages (1.130.0)

Requirement already satisfied: google-cloud-storage in /usr/local/lib/python3.12/dist-packages (3.7.0)

Requirement already satisfied: google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,!=2.4.*,!=2.5.*,!=2.6.*,!=2.7.*,<3.0.0,>=1.34.1 in /usr/local/lib/python3.12/dist-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,!=2.4.*,!=2.5.*,!=2.6.*,!=2.7.*,<3.0.0,>=1.34.1->google-cloud-aiplatform) (2.28.0)

Requirement already satisfied: google-auth<3.0.0,>=2.14.1 in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (2.38.0)

Requirement already satisfied: proto-plus<2.0.0,>=1.22.3 in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (1.26.1)

Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<7.0.0,>=3.20.2 in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (6.33.0)

Requirement already satisfied: packaging>=14.3 in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (25.0)

Requirement already satisfied: google-cloud-bigquery!=3.20.0,<4.0.0,>=1.15.0 in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (3.38.0)

Requirement already satisfied: google-cloud-resource-manager<3.0.0,>=1.3.3 in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (1.15.0)

Requirement already satisfied: shapely<3.0.0 in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (2.1.2)

Requirement already satisfied: google-genai<2.0.0,>=1.37.0 in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (1.46.0)

Requirement already satisfied: pydantic<3 in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (2.11.10)

Requirement already satisfied: typing_extensions in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (4.15.0)

Requirement already satisfied: docstring_parser<1 in /usr/local/lib/python3.12/dist-packages (from google-cloud-aiplatform) (0.17.0)

Requirement already satisfied: google-cloud-core<3.0.0,>=2.4.2 in /usr/local/lib/python3.12/dist-packages (from google-cloud-storage) (2.4.3)

Requirement already satisfied: google-resumable-media<3.0.0,>=2.7.2 in /usr/local/lib/python3.12/dist-packages (from google-cloud-storage) (2.7.2)

Requirement already satisfied: requests<3.0.0,>=2.22.0 in /usr/local/lib/python3.12/dist-packages (from google-cloud-storage) (2.32.4)

Requirement already satisfied: google-crc32c<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from google-cloud-storage) (1.7.1)

Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.56.2 in /usr/local/lib/python3.12/dist-packages (from google-api-core!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,!=2.4.*,!=2.5.*,!=2.6.*,!=2.7.*,<3.0.0,>=1.34.1->google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,!=2.4.*,!=2.5.*,!=2.6.*,!=2.7.*,<3.0.0,>=1.34.1->google-cloud-aiplatform) (1.71.0)

Requirement already satisfied: grpcio<2.0.0,>=1.33.2 in /usr/local/lib/python3.12/dist-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,!=2.4.*,!=2.5.*,!=2.6.*,!=2.7.*,<3.0.0,>=1.34.1->google-cloud-aiplatform) (1.76.0)

Requirement already satisfied: grpcio-status<2.0.0,>=1.33.2 in /usr/local/lib/python3.12/dist-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,!=2.4.*,!=2.5.*,!=2.6.*,!=2.7.*,<3.0.0,>=1.34.1->google-cloud-aiplatform) (1.76.0)

Requirement already satisfied: cachetools<6.0,>=2.0.0 in /usr/local/lib/python

on3.12/dist-packages (from google-auth<3.0.0,>=2.14.1->google-cloud-aiplatform) (5.5.2)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.0,>=2.14.1->google-cloud-aiplatform) (0.4.2)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.0,>=2.14.1->google-cloud-aiplatform) (4.9.1)
Requirement already satisfied: python-dateutil<3.0.0,>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from google-cloud-bigquery!=3.20.0,<4.0.0,>=1.15.0->google-cloud-aiplatform) (2.9.0.post0)
Requirement already satisfied: grpc-google-iam-v1<1.0.0,>=0.14.0 in /usr/local/lib/python3.12/dist-packages (from google-cloud-resource-manager<3.0.0,>=1.3.3->google-cloud-aiplatform) (0.14.3)
Requirement already satisfied: anyio<5.0.0,>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0,>=1.37.0->google-cloud-aiplatform) (4.11.0)
Requirement already satisfied: httpx<1.0.0,>=0.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0,>=1.37.0->google-cloud-aiplatform) (0.28.1)
Requirement already satisfied: tenacity<9.2.0,>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0,>=1.37.0->google-cloud-aiplatform) (8.5.0)
Requirement already satisfied: websockets<15.1.0,>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0,>=1.37.0->google-cloud-aiplatform) (15.0.1)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3->google-cloud-aiplatform) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3->google-cloud-aiplatform) (2.33.2)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3->google-cloud-aiplatform) (0.4.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.22.0->google-cloud-storage) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.22.0->google-cloud-storage) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.22.0->google-cloud-storage) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.22.0->google-cloud-storage) (2025.10.5)
Requirement already satisfied: numpy>=1.21 in /usr/local/lib/python3.12/dist-packages (from shapely<3.0.0->google-cloud-aiplatform) (1.26.4)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0.0,>=4.8.0->google-genai<2.0.0,>=1.37.0->google-cloud-aiplatform) (1.3.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->google-genai<2.0.0,>=1.37.0->google-cloud-aiplatform) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0.0,>=0.28.1->google-genai<2.0.0,>=1.37.0->google-cloud-aiplatform) (0.16.0)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->google-auth<3.0.0,>=2.14.1->

google-cloud-aiplatform) (0.6.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil<3.0.0,>=2.8.2->google-cloud-bigquery!=3.20.0,<4.0.0,>=1.15.0->google-cloud-aiplatform) (1.17.0)
Requirement already satisfied: scikit-learn==1.3.2 in /usr/local/lib/python3.12/dist-packages (1.3.2)
Requirement already satisfied: numpy<2.0,>=1.17.3 in /usr/local/lib/python3.12/dist-packages (from scikit-learn==1.3.2) (1.26.4)
Requirement already satisfied: scipy>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn==1.3.2) (1.16.3)
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from scikit-learn==1.3.2) (1.5.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn==1.3.2) (3.6.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.2)
Requirement already satisfied: transformers==4.46.1 in /usr/local/lib/python3.12/dist-packages (4.46.1)
Requirement already satisfied: accelerate>=0.33.0 in /usr/local/lib/python3.12/dist-packages (1.12.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.1) (3.20.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.23.2 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.1) (0.36.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.1) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.1) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.1) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.1) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.1) (2.32.4)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.1) (0.6.2)
Requirement already satisfied: tokenizers<0.21,>=0.20 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.1) (0.20.3)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.1) (4.67.1)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate>=0.33.0) (5.9.5)
Requirement already satisfied: torch>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from accelerate>=0.33.0) (2.9.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.23.2->transformers==4.46.1) (2025.3.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.23.2->transformers==4.46.1) (4.15.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.23.2->transformers==4.46.1) (1.2.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/di

st-packages (from torch>=2.0.0->accelerate>=0.33.0) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/
dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (3.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-pack
ages (from torch>=2.0.0->accelerate>=0.33.0) (3.1.6)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/loca
l/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (12.
8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/loc
al/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (12.
8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/loca
l/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (12.
8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/li
b/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (9.10.2.2
1)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/li
b/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (12.8.4.
1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/li
b/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (11.3.3.8
3)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/l
ib/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (10.3.9.
90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/loca
l/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (11.
7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/loca
l/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (12.
5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/l
ib/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/py
thon3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/li
b/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (3.3.20)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/p
ython3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/
lib/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (12.8.9
3)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/li
b/python3.12/dist-packages (from torch>=2.0.0->accelerate>=0.33.0) (1.13.1.
3)
Requirement already satisfied: triton==3.5.1 in /usr/local/lib/python3.12/di
st-packages (from torch>=2.0.0->accelerate>=0.33.0) (3.5.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/py
thon3.12/dist-packages (from requests->transformers==4.46.1) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dis
t-packages (from requests->transformers==4.46.1) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.
12/dist-packages (from requests->transformers==4.46.1) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.
12/dist-packages (from requests->transformers==4.46.1) (2025.10.5)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=2.0.0->accelerate>=0.33.0) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=2.0.0->accelerate>=0.33.0) (3.0.3)

Requirement already satisfied: torch>=2.2.0 in /usr/local/lib/python3.12/dist-packages (2.9.1)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (3.20.0)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (4.15.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (1.13.3)

Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (3.5)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (2025.3.0)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (12.8.93)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (12.8.90)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (12.8.90)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (12.8.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (11.3.3.83)

Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (10.3.9.90)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (11.7.3.90)

Requirement already satisfied: nvidia-cusparselt-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (12.5.8.93)

Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (12.5.8.93)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (2.27.5)

Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (3.3.20)

Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (12.8.90)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (12.8.93)

Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (1.13.1.3)

Requirement already satisfied: triton==3.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2.0) (3.5.1)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=2.2.0) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/

dist-packages (from jinja2->torch>=2.2.0) (3.0.3)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.3.3)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

1. Imports and Global Config

```
In [10]: import os
import time
import json
from pathlib import Path

import numpy as np
import pandas as pd

import torch
from torch import nn
from torch.quantization import quantize_dynamic

from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import joblib

from transformers import AutoModelForCausalLM, AutoTokenizer

from google.cloud import aiplatform
from google.cloud import storage
```

Configure GCP project

```
In [11]: PROJECT_ID = "instr-cs795-fall25-hqin-1"
REGION = "us-central1" # or the region you used in class
BUCKET_NAME = "instr-cs795-fall25-hqin-1-arasm002" # without gs://

STAGING_BUCKET_URI = f"gs://{BUCKET_NAME}/vertex_staging"
EXPERIMENT_ROOT = Path("./experiments")
EXPERIMENT_ROOT.mkdir(exist_ok=True, parents=True)

aiplatform.init(
    project=PROJECT_ID,
    location=REGION,
    staging_bucket=STAGING_BUCKET_URI,
)
```

```
storage_client = storage.Client(project=PROJECT_ID)
```

2. Vertex AI: Iris RandomForest End-to-End

This section reproduces:

- Local training
- Upload to GCS
- Vertex AI Model Registry registration
- Endpoint deployment & online prediction
- Batch prediction job

```
In [12]: IRIS_ARTIFACT_DIR = EXPERIMENT_ROOT / "iris_model"
IRIS_ARTIFACT_DIR.mkdir(exist_ok=True, parents=True)

MODEL_FILENAME = IRIS_ARTIFACT_DIR / "model.joblib"

# Train the local model
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
)
rf.fit(X_train, y_train)

acc = rf.score(X_test, y_test)
print(f"Iris RandomForest test accuracy: {acc:.4f}")

joblib.dump(rf, MODEL_FILENAME)
print(f"Saved model to {MODEL_FILENAME}")
```

Iris RandomForest test accuracy: 1.0000

Saved model to experiments/iris_model/model.joblib

```
In [13]: # Upload artifacts to GCS
artifact_gcs_uri = f"gs://{BUCKET_NAME}/iris_model"
bucket = storage_client.bucket(BUCKET_NAME)
blob = bucket.blob("iris_model/model.joblib")
blob.upload_from_filename(str(MODEL_FILENAME))
print(f"Uploaded model artifact to {artifact_gcs_uri}")
```

Uploaded model artifact to gs://instr-cs795-fall25-hqin-1-arasm002/iris_model

```
In [14]: import sklearn
print(f"Notebook Scikit-learn version: {sklearn.__version__}")
```

Notebook Scikit-learn version: 1.3.2

```
In [15]: # Register the model in Vertex AI Model Registry

from google.cloud import aiplatform

MODEL_DISPLAY_NAME = "iris-rf-sklearn-1-3"
SERVING_IMAGE = "us-docker.pkg.dev/vertex-ai/prediction/sklearn-cpu.1-3:late

vertex_model = aiplatform.Model.upload(
    display_name=MODEL_DISPLAY_NAME,
    artifact_uri=artifact_gcs_uri,
    serving_container_image_uri=SERVING_IMAGE,
)

print("Uploaded Vertex AI model:")
print(vertex_model.resource_name)
```

Uploaded Vertex AI model:
projects/104115398803/locations/us-central1/models/5825871211422285824

```
In [16]: # Avoid re-running as this takes too long

from google.cloud import aiplatform

# 1. SETUP: Use a FRESH name to avoid locks
ENDPOINT_DISPLAY_NAME = "iris-rf-endpoint-fresh-v2"

try:
    print(f"Creating new endpoint: {ENDPOINT_DISPLAY_NAME}")
    endpoint = aiplatform.Endpoint.create(display_name=ENDPOINT_DISPLAY_NAME)
    print(f"Created: {endpoint.resource_name}")

    # 2. DEPLOY
    print("Starting deployment (this takes ~10-15 mins)...")
    deployed_model = vertex_model.deploy(
        endpoint=endpoint,
        machine_type="n1-standard-4",
        traffic_split={"0": 100},
        sync=True
    )
    print("✅ Model deployed successfully!")

    # 3. TEST
    print("Sending test prediction...")
    # Iris dataset sample: [sepal_len, sepal_width, petal_len, petal_width]
    test_prediction = endpoint.predict(instances=[[5.1, 3.5, 1.4, 0.2]])
    print("Prediction Result:", test_prediction.predictions)

except Exception as e:
    print(f"❌ Deployment failed: {e}")

finally:
    # 4. CLEANUP (Optional)
    # If you want to keep it running to show your instructor, COMMENT OUT the
    print("\n--- Cleanup ---")
    # if 'endpoint' in locals():
    #     print("Undeploying and deleting to save cost...")
```

```
# endpoint.undeploy_all()
# endpoint.delete()
# print("Deleted.")
```

Creating new endpoint: iris-rf-endpoint-fresh-v2

Created: projects/104115398803/locations/us-central1/endpoints/6174917774729543680

Starting deployment (this takes ~10-15 mins)...

✓ Model deployed successfully!

Sending test prediction...

Prediction Result: [0.0]

--- Cleanup ---

```
In [17]: from google.cloud import aiplatform

aiplatform.init(project=PROJECT_ID, location=REGION)

endpoints = aiplatform.Endpoint.list()

for ep in endpoints:
    print("Display name:", ep.display_name)
    print("Resource name:", ep.resource_name)
    print("Traffic split:", ep.traffic_split)
    print("-" * 60)
```

Display name: iris-rf-endpoint-fresh-v2

Resource name: projects/104115398803/locations/us-central1/endpoints/6174917774729543680

Traffic split: {'8755735447910481920': 100}

Display name: iris-rf-endpoint-fresh-v2

Resource name: projects/104115398803/locations/us-central1/endpoints/4628212777704488960

Traffic split: {'1261745667965976576': 100}

Display name: llama-3-2-3b-instruct-mg-one-click-deploy

Resource name: projects/104115398803/locations/us-central1/endpoints/mg-endpoint-02a55366-2876-499b-8577-9e439cffffa60

Traffic split: {'4816757031635517440': 100}

```
In [18]: ENDPOINT_DISPLAY_NAME = "iris-rf-endpoint-fresh-v2"

# replace with the resource name of the healthy endpoint
ENDPOINT_NAME = "projects/104115398803/locations/us-central1/endpoints/4628212777704488960"

endpoint = aiplatform.Endpoint(ENDPOINT_NAME)

# sanity check
print("Endpoint:", endpoint.resource_name)
print("Traffic split:", endpoint.traffic_split)

# now retry online prediction
sample_instance = X_test[0].tolist()
prediction = endpoint.predict(instances=[sample_instance])
print("Online prediction:", prediction.predictions, "ground truth:", int(y_test[0]))
```

Endpoint: projects/104115398803/locations/us-central1/endpoints/4628212777704488960
Traffic split: {'1261745667965976576': 100}
Online prediction: [1.0] ground truth: 1

```
In [19]: # Test online prediction
sample_instance = X_test[0].tolist()
request_instances = [sample_instance]

prediction = endpoint.predict(instances=request_instances)
print("Online prediction:", prediction.predictions, "ground truth:", int(y_test[0]))
```

Online prediction: [1.0] ground truth: 1

```
In [20]: import json

# Batch prediction setup
BATCH_INPUT_GCS = f"gs://{BUCKET_NAME}/iris_batch_input.jsonl"
BATCH_OUTPUT_GCS = f"gs://{BUCKET_NAME}/iris_batch_output.jsonl"

# Create simple JSONL input locally and upload
# Each line = one instance, same shape as you send to endpoint.predict
local_batch_file = EXPERIMENT_ROOT / "iris_batch_input.jsonl"
with open(local_batch_file, "w") as f:
    for x in X_test[:10]:
        instance = x.tolist()
        f.write(json.dumps(instance) + "\n")

batch_blob = bucket.blob("iris_batch_input.jsonl")
batch_blob.upload_from_filename(str(local_batch_file))
print(f"Uploaded batch input to {BATCH_INPUT_GCS}")
```

Uploaded batch input to gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_input.jsonl

```
In [21]: batch_job = vertex_model.batch_predict(
    job_display_name="iris-rf-batch-prediction-fresh-v2",
    instances_format="jsonl",
    predictions_format="jsonl",
    gcs_source=[BATCH_INPUT_GCS],
    gcs_destination_prefix=BATCH_OUTPUT_GCS,
```



```
    machine_type="n1-standard-2",  
    sync=True,  
)  
  
print("Batch prediction job resource:", batch_job.resource_name)  
print("Final state:", batch_job.state)  
print("Error:", batch_job.error)
```

Batch prediction job resource: projects/104115398803/locations/us-central1/batchPredictionJobs/8051297366109585408
Final state: 4
Error:

```
In [22]: BATCH_OUTPUT_GCS = f"gs://{BUCKET_NAME}/iris_batch_output"  
!gsutil ls {BATCH_OUTPUT_GCS}  
!gsutil ls {BATCH_OUTPUT_GCS}/*/  
!gsutil cat {BATCH_OUTPUT_GCS}/*/prediction.results-* | head
```

```

gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T18_52_18_476Z/
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T19_10_42_344Z/
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T19_44_40_802Z/
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T20_46_08_719Z/
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T18_52_18_476Z/:
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T18_52_18_476Z/prediction.errors_stats-00000-of-00001
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T18_52_18_476Z/prediction.results-00000-of-00001

gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T19_10_42_344Z/:
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T19_10_42_344Z/prediction.errors_stats-00000-of-00001
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T19_10_42_344Z/prediction.results-00000-of-00001

gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T19_44_40_802Z/:
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T19_44_40_802Z/prediction.errors_stats-00000-of-00001
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T19_44_40_802Z/prediction.results-00000-of-00001

gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T20_46_08_719Z/:
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T20_46_08_719Z/prediction.errors_stats-00000-of-00001
gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf
-sklearn-1-3-2025_12_10T20_46_08_719Z/prediction.results-00000-of-00001
{"instance": [6.1, 2.8, 4.7, 1.2], "prediction": 1}
{"instance": [5.7, 3.8, 1.7, 0.3], "prediction": 0}
{"instance": [7.7, 2.6, 6.9, 2.3], "prediction": 2}
{"instance": [6.0, 2.9, 4.5, 1.5], "prediction": 1}
{"instance": [6.8, 2.8, 4.8, 1.4], "prediction": 1}
{"instance": [5.4, 3.4, 1.5, 0.4], "prediction": 0}
{"instance": [5.6, 2.9, 3.6, 1.3], "prediction": 1}
{"instance": [6.9, 3.1, 5.1, 2.3], "prediction": 2}
{"instance": [6.2, 2.2, 4.5, 1.5], "prediction": 1}
{"instance": [5.8, 2.7, 3.9, 1.2], "prediction": 1}
Exception ignored in: <_io.TextIOWrapper name='<stdout>' mode='w' encoding
='utf-8'>
BrokenPipeError: [Errno 32] Broken pipe

```

```

In [23]: from google.cloud import storage
import re

storage_client = storage.Client()

prefix = "iris_batch_output/"
blobs = list(storage_client.list_blobs(BUCKET_NAME, prefix=prefix))

```

```

# Extract the unique job directories from blob names
job_dirs = set()

for blob in blobs:
    # blob.name looks like:
    # iris_batch_output/prediction-iris-rf-sklearn-.../prediction.results-
    m = re.match(r"iris_batch_output/([^/]+)/", blob.name)
    if m:
        job_dirs.add(m.group(1))

job_dirs = sorted(job_dirs)
print("Job directories found:", job_dirs)

latest_dir = job_dirs[-1] # pick the most recent
print("Latest job directory:", latest_dir)

RESULTS_GCS = f"gs://{BUCKET_NAME}/iris_batch_output/{latest_dir}/prediction
print("Results file:", RESULTS_GCS)

local_results = EXPERIMENT_ROOT / "vertex_batch_results.jsonl"
!gsutil cp {RESULTS_GCS} {local_results}

records = []
with open(local_results, "r") as f:
    for line in f:
        records.append(json.loads(line))

import pandas as pd
pred_df = pd.DataFrame(records)
pred_df

```

Job directories found: ['prediction-iris-rf-sklearn-1-3-2025_12_10T18_52_18_476Z', 'prediction-iris-rf-sklearn-1-3-2025_12_10T19_10_42_344Z', 'prediction-iris-rf-sklearn-1-3-2025_12_10T19_44_40_802Z', 'prediction-iris-rf-sklearn-1-3-2025_12_10T20_46_08_719Z']

Latest job directory: prediction-iris-rf-sklearn-1-3-2025_12_10T20_46_08_719Z

Results file: gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf-sklearn-1-3-2025_12_10T20_46_08_719Z/prediction.results-00000-of-00001

Copying gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output/prediction-iris-rf-sklearn-1-3-2025_12_10T20_46_08_719Z/prediction.results-00000-of-00001...

/ [1 files][520.0 B/ 520.0 B]

Operation completed over 1 objects/520.0 B.

Out [23]:

	instance	prediction
0	[6.1, 2.8, 4.7, 1.2]	1
1	[5.7, 3.8, 1.7, 0.3]	0
2	[7.7, 2.6, 6.9, 2.3]	2
3	[6.0, 2.9, 4.5, 1.5]	1
4	[6.8, 2.8, 4.8, 1.4]	1
5	[5.4, 3.4, 1.5, 0.4]	0
6	[5.6, 2.9, 3.6, 1.3]	1
7	[6.9, 3.1, 5.1, 2.3]	2
8	[6.2, 2.2, 4.5, 1.5]	1
9	[5.8, 2.7, 3.9, 1.2]	1

In [24]: `import numpy as np`

```
# Compare predictions against the first 10 y_test labels
y_true_10 = y_test[:len(pred_df)]
y_pred_10 = np.array(pred_df["prediction"].tolist())

accuracy_10 = (y_true_10 == y_pred_10).mean()
print(f"Batch prediction accuracy on 10 Iris instances: {accuracy_10:.2f}")

print("Vertex AI batch inference summary:")
print("Batch input:", BATCH_INPUT_GCS)
print("Batch output prefix:", BATCH_OUTPUT_GCS)
print("Latest job dir:", latest_dir)
print(f"Number of instances scored: {len(pred_df)}")
print(f"Sample prediction row:\n{pred_df.head(1)}")
```

Batch prediction accuracy on 10 Iris instances: 1.00

Vertex AI batch inference summary:

Batch input: gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_input.jsonl

Batch output prefix: gs://instr-cs795-fall25-hqin-1-arasm002/iris_batch_output

Latest job dir: prediction-iris-rf-sklearn-1-3-2025_12_10T20_46_08_719Z

Number of instances scored: 10

Sample prediction row:

	instance	prediction
0	[6.1, 2.8, 4.7, 1.2]	1

3. Dynamic INT8 Quantization (Toy Model Demo)

To keep this notebook reproducible in a constrained runtime, we demonstrate PyTorch dynamic INT8 quantization on a tiny feedforward network instead of a full transformer LLM. The same quantization API applies to larger models (e.g., DistilGPT2) used in prior milestones.

```
In [25]: import time
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.quantization as tq

DEVICE = "cpu"
print("Using device:", DEVICE)

metrics_rows = []
```

Using device: cpu

```
In [26]: class TinyMLP(nn.Module):
    def __init__(self, in_dim=16, hidden_dim=32, out_dim=16):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, out_dim),
        )

    def forward(self, x):
        return self.net(x)

def benchmark_tiny_model(model, in_dim=16, runs=100):
    model.eval()
    x = torch.randn(1, in_dim)

    # Warmup
    with torch.no_grad():
        _ = model(x)

    times = []
    with torch.no_grad():
        for _ in range(runs):
            start = time.time()
            _ = model(x)
            end = time.time()
            times.append(end - start)

    return float(np.mean(times))
```

```
def param_size_mb(model, bytes_per_param=4):
    total_params = sum(p.numel() for p in model.parameters())
    return total_params * bytes_per_param / (1024 * 1024)
```

Run FP32 vs INT8 on the toy model

```
In [27]: # FP32 baseline
fp_model = TinyMLP()
fp_latency = benchmark_tiny_model(fp_model)
fp_size_mb = param_size_mb(fp_model, bytes_per_param=4)

print(f"[TinyMLP FP32] latency: {fp_latency*1e3:.3f} ms, size≈{fp_size_mb:.4} MB")

metrics_rows.append({
    "model": "TinyMLP",
    "variant": "fp32",
    "quant_method": "none",
    "latency_s_forward": fp_latency,
    "size_mb": fp_size_mb,
})

# Dynamic INT8 quantization (Linear layers)
int8_model = tq.quantize_dynamic(
    fp_model,
    {nn.Linear},
    dtype=torch.qint8,
)

int8_latency = benchmark_tiny_model(int8_model)
# Rough size: treat quantized params as 1 byte each
int8_size_mb = param_size_mb(int8_model, bytes_per_param=1)

print(f"[TinyMLP INT8] latency: {int8_latency*1e3:.3f} ms, size≈{int8_size_mb:.4} MB")

metrics_rows.append({
    "model": "TinyMLP",
    "variant": "int8_dynamic",
    "quant_method": "dynamic_int8",
    "latency_s_forward": int8_latency,
    "size_mb": int8_size_mb,
})

metrics_df = pd.DataFrame(metrics_rows)
metrics_df
```

[TinyMLP FP32] latency: 0.042 ms, size≈0.0041 MB

[TinyMLP INT8] latency: 0.147 ms, size≈0.0000 MB

```

/tmp/ipython-input-2435141178.py:17: DeprecationWarning: torch.ao.quantization
is deprecated and will be removed in 2.10.
For migrations of users:
1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantization.quantize_dynamic), please migrate to use torchao eager mode quantize_API instead
2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx, torch.ao.quantization.quantize_fx.convert_fx, please migrate to use torchao pt2e quantization API instead (prepare_pt2e, convert_pt2e)
3. pt2e quantization has been migrated to torchao (https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e)
see https://github.com/pytorch/ao/issues/2259 for more details
    int8_model = tq.quantize_dynamic(

```

```

Out[27]:

```

	model	variant	quant_method	latency_s_forward	size_mb
0	TinyMLP	fp32	none	0.000042	0.004089
1	TinyMLP	int8_dynamic	dynamic_int8	0.000147	0.000000

4. Federated Learning Prototype (FedAvg)

Minimal FedAvg on synthetic client updates, serving as a placeholder for future decentralized LLM training experiments.

```

In [28]: def fedavg(updates: np.ndarray) -> np.ndarray:
        """Simple FedAvg: mean over client updates."""
        return updates.mean(axis=0)

client_updates = np.array([
    [1.0, 2.0, 3.0],
    [1.2, 1.9, 3.1],
    [0.9, 2.1, 2.9],
    [1.1, 2.2, 3.2],
])

global_update = fedavg(client_updates)
print("Client updates:\n", client_updates)
print("Global update:", global_update)

```

```

Client updates:
[[1.  2.  3.]
 [1.2 1.9 3.1]
 [0.9 2.1 2.9]
 [1.1 2.2 3.2]]
Global update: [1.05 2.05 3.05]

```

```

In [29]: metrics_df = pd.concat([
        metrics_df,
        pd.DataFrame([
            {"model": "fedavg_proto",
             "variant": "baseline",
             "quant_method": "n/a",

```

```

        "latency_s_forward": None,
        "size_mb": None,
    })
], ignore_index=True)

metrics_df

```

/tmp/ipython-input-4008146550.py:1: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA entries is deprecated. In a future version, this will no longer exclude empty or all-NA columns when determining the result dtypes. To retain the old behavior, exclude the relevant entries before the concat operation.

```

    metrics_df = pd.concat([

```

Out [29]:

	model	variant	quant_method	latency_s_forward	size_mb
0	TinyMLP	fp32	none	0.000042	0.004089
1	TinyMLP	int8_dynamic	dynamic_int8	0.000147	0.000000
2	fedavg_proto	baseline	n/a	NaN	NaN

5. Save Metrics for Analysis / Paper Tables

In [30]:

```

from pathlib import Path

EXPERIMENT_ROOT = Path("./experiments")
EXPERIMENT_ROOT.mkdir(exist_ok=True, parents=True)

METRICS_CSV = EXPERIMENT_ROOT / "metrics_tinymlp_fedavg.csv"
metrics_df.to_csv(METRICS_CSV, index=False)
print("Saved metrics to", METRICS_CSV)
metrics_df

```

Saved metrics to experiments/metrics_tinymlp_fedavg.csv

Out [30]:

	model	variant	quant_method	latency_s_forward	size_mb
0	TinyMLP	fp32	none	0.000042	0.004089
1	TinyMLP	int8_dynamic	dynamic_int8	0.000147	0.000000
2	fedavg_proto	baseline	n/a	NaN	NaN

6. GPTQ and AWQ Quantization (Theoretical Pipelines)

The next phase of the project explores *model compression techniques* suitable for deploying transformer models on edge devices. While the notebook environment cannot execute full GPTQ/AWQ quantization on multi-billion-parameter models, it is still

essential to document the pipeline, algorithms, and configuration patterns that underpin these methods.

Accordingly, this section presents:

- A **theoretical GPTQ pipeline** for a medium-scale model (Phi-2),
- A **theoretical AWQ pipeline**, and
- Configurations consistent with current literature and open-source toolkits.

These serve as **methodological blueprints** for future implementation once a suitable hardware environment (e.g., GPU-enabled research node) is available. The resulting descriptions align with the approach motivating the project's long-term vision of edge-ready LLMs for healthcare privacy.

6.1 GPTQ Quantization Pipeline (Conceptual Example for Phi-2)

The following GPTQ pipeline illustrates how a 4-bit compressed Phi-2 model would be produced:

```
from transformers import AutoTokenizer
from auto_gptq import AutoGPTQForCausalLM, BaseQuantizeConfig

MODEL = "microsoft/phi-2"

# Configure GPTQ for 4-bit quantization
quant_cfg = BaseQuantizeConfig(
    bits=4,
    group_size=128,
    desc_act=False,
)

tokenizer = AutoTokenizer.from_pretrained(MODEL, use_fast=True)

# Load model with GPTQ wrapper
model = AutoGPTQForCausalLM.from_pretrained(
    MODEL,
    quantize_config=quant_cfg,
)

# Conceptual calibration pass
model.quantize(
    calibration_dataset="wikitext2",
    max_input_length=512,
)

# Save the quantized weights
model.save_quantized("phi2-gptq-4bit")
```

6.3 Loading Offline (or Hypothetical) GPTQ/AWQ Results

In a full implementation, you might export quantization results to a CSV file:

```
model,method,bits,group_size,size_mb,latency_s_32_tokens,perplexity
Phi-2,GPTQ,4,128,1500,0.85,27.4
Phi-2,AWQ,4,128,1520,0.90,27.8
```

You can display these results inside the notebook using:

```
import pandas as pd
pd.read_csv("phi2_quant_results.csv")
```

7. GGUF Export & llama.cpp Inference (Conceptual)

To support CPU-only or embedded devices, quantized models are often exported to GGUF, the format used by llama.cpp.

This section documents the edge-deployment workflow conceptually.

7.1 Conceptual GGUF Export Pipeline

```
from transformers import AutoModelForCausalLM
from gguf import GGUFWriter  # conceptual placeholder API

# Load previously quantized model (e.g., GPTQ)
model = AutoModelForCausalLM.from_pretrained("phi2-gptq-4bit")

# Prepare GGUF writer
writer = GGUFWriter("phi2-gptq.gguf", "Phi-2-GPTQ")

# Add parameters to GGUF archive
for name, tensor in model.state_dict().items():
    writer.add_tensor(name, tensor.cpu().numpy())

# Finalize GGUF file
writer.write_header_to_file()
writer.close()
```

7.2 Conceptual llama.cpp Benchmark Metrics

Typical measurements include:

Tokens per second

RAM footprint

Thread count

Example benchmark table:

```
model,tokens_per_second,ram_mb
Phi2-GPTQ-4bit.gguf,18.5,1450
Phi2-AWQ-4bit.gguf,17.9,1475
```

To visualize:

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("llama_cpp_benchmarks.csv")
df.plot.bar(x="model", y="tokens_per_second", title="Edge Inference
Throughput")
```

8. Federated Learning Prototype (Runnable Conceptual Example)

This section demonstrates the mechanics of federated averaging using a tiny neural network.

It provides a conceptual stand-in for how a quantized LLM might be updated across devices in a privacy-preserving system.

8.1 Tiny Model + FedAvg Algorithm (Readable Markdown Version)

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

# Simple classifier to simulate a "local model"
class TinyClassifier(nn.Module):
    def __init__(self, in_dim=4, hidden_dim=8, out_dim=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, out_dim),
        )

    def forward(self, x):
        return self.net(x)

# Flatten model parameters into a vector
def get_params_vector(model):
```

```

        return torch.cat([p.data.view(-1) for p in model.parameters()])

# Set model parameters from vector
def set_params_vector(model, vec):
    idx = 0
    for p in model.parameters():
        numel = p.numel()
        p.data.copy_(vec[idx:idx+numel].view_as(p))
        idx += numel

# Federated averaging over parameter vectors
def fedavg(vectors):
    return torch.stack(vectors).mean(dim=0)

```

8.2 Simulate Three Client Updates + Aggregation

```

torch.manual_seed(42)

global_model = TinyClassifier()
global_params = get_params_vector(global_model)

num_clients = 3
client_param_vectors = []

# Synthetic data
X = torch.randn(16, 4)
y = torch.randint(0, 3, (16,))

# Each client performs one gradient step
for i in range(num_clients):
    model = TinyClassifier()
    set_params_vector(model, global_params.clone())

    opt = optim.SGD(model.parameters(), lr=0.1)
    loss = nn.CrossEntropyLoss()(model(X), y)
    loss.backward()
    opt.step()

    client_param_vectors.append(get_params_vector(model))

# Aggregate client weights
new_global_params = fedavg(client_param_vectors)
set_params_vector(global_model, new_global_params)

```

8.3 Global Model Sanity Check

```

global_model.eval()
with torch.no_grad():
    preds = global_model(X).argmax(dim=1)
    acc = (preds == y).float().mean().item()

```

acc

9. Final Summary

This notebook demonstrates:

Executed Work

- Full Vertex AI deployment pipeline
- Online and batch inference on Iris
- Working INT8 quantization demo (TinyMLP)
- Runnable federated learning prototype

Theoretical Work

- GPTQ quantization pipeline
- AWQ quantization pipeline
- GGUF edge deployment pipeline
- Conceptual llama.cpp benchmarks

Together, these components outline a hybrid cloud-edge framework for privacy-preserving LLM inference in healthcare.